

ФКН ВШЭ

Глубинное обучение 1, 2024/25

Автор конспекта: Даниил Тихонов

27 марта 2025 г. в 00:44

Содержание

1. Вариационный автокодировщик (VAE)	1
1.1. Manifold Hypothesis	1
1.2. Понижение размерности	1
1.3. Autoencoder	2
1.4. Denoising Autoencoder	3
1.5. Семплирование	3
1.6. Variational Autoencoder (VAE)	4
1.7. Reparameterization trick	8
1.8. Вывод формулы итогового лосса	8
1.9. Заключение	9

Лекция #14: Вариационный автокодировщик (VAE)

27 февраля, 2025

1.1. Manifold Hypothesis

Прежде чем начинать говорить об автокодировщиках (автоэнкодерах), стоит обосновать законность дальнейших действий, и почему это должно теоретически работать.

Manifold Hypothesis

Гипотеза гласит о том, что бо́льшая часть присущих реальному миру многомерных данных (картинки, тексты, видео, звук, графы и т.д.) в действительности лежит на низкоразмерном многообразии живущем в многомерном пространстве.

Другими словами, несмотря на то, что очень часто данные описываются огромным количеством разных признаков (комбинации пикселей на картинке, n -грамм в тексте), в реальности только небольшое и хорошо структурированное подмножество комбинаций соответствует настоящим данным, а остальную часть пространства входов составляют нереалистичные объекты.

Заметим, что данное знание можно использовать для задачи понижения размерности. Исходя из истинности гипотезы можно построить такое отображение в маломерное пространство признаков, что все свойства объектов в нем сохраняются.

1.2. Понижение размерности

Пусть имеются многомерные объекты $x \in \mathbb{R}^D$, которые мы хотим отобразить в скрытые (латентные) представления $z \in \mathbb{R}^d$, то есть найти

$$f : \mathbb{R}^D \rightarrow \mathbb{R}^d, \quad f(x) = z, \quad d < D \quad (d \ll D)$$

Для начала рассмотрим простой случай, когда отображение f линейно. Имея в нашем распоряжении N объектов для обучения, удобно представить задачу в матричном виде:

$$X \in \mathbb{R}^{N \times D}, \quad Z \in \mathbb{R}^{N \times d} \implies f(X) = X\Theta = Z, \quad \text{где } \Theta \in \mathbb{R}^{D \times d}$$

Осталось поставить задачу поиска Θ так, чтобы латентные представления Z были действительно информативными. Для этого естественно ввести дополнительное ограничение: мы хотим уметь реконструировать исходные объекты X по их скрытым представлениям Z :

$$\hat{X} = Z\Phi, \quad \text{где } \Phi \in \mathbb{R}^{d \times D}$$

Взяв в качестве лосса обычный MSE, задача обучения принимает вид

$$\text{MSE}(X, \hat{X}) = \|X - \hat{X}\|_F^2 = \|X - Z\Phi\|_F^2 = \|X - X\Theta\Phi\|_F^2 \longrightarrow \min_{\Theta, \Phi}$$

$$A = (a_{ij})_{i,j=1}^{n,m} \implies \|A\|_F = \left(\sum_{i,j=1}^{n,m} |a_{ij}|^2 \right)^{1/2}$$

Заметим, что данная задача сводится к малоранговому приближению матрицы:

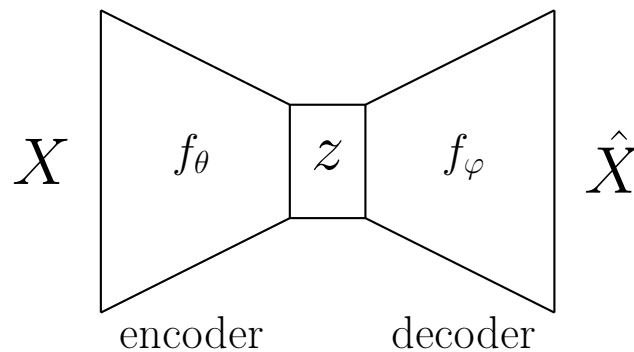
$$\hat{X} = X \underbrace{\Theta}_{d \times D} \underbrace{\Phi}_{D \times d} \implies rk(\hat{X}) = rk(X\Theta\Phi) \leq d$$

В сущности это называется методом главных компонент (PCA) и решается аналитически.

1.3. Autoencoder

Теперь рассмотрим более сложный случай, где уже нет понятного решения. Если быть конкретнее, разрешим отображению f быть нелинейным, а именно параметризоваться нейросетью.

Схема кодирования и декодирования будет иметь следующую bottleneck структуру:

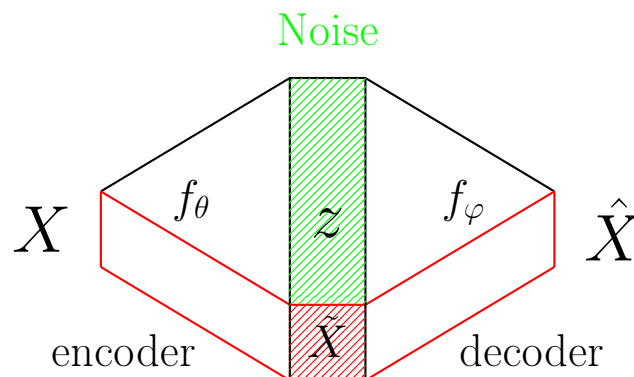


Тут f_θ , f_φ – нейросети с параметрами θ и φ соответственно. f_θ называется **энкодером**, так как кодирует исходные объекты x в латентные представления z . f_φ называется **декодером**, так как восстанавливает из латентного представления z исходный объект x .

Обозначим за $\mathcal{X}$ распределение наших данных (в реальности конечная выборка). Тогда лосс, на который мы будем учить модель, будет очень похож на MSE:

$$\mathbb{E}_{x \sim \mathcal{X}} \left[\|x - \hat{x}\|_2^2 \right] = \mathbb{E}_{x \sim \mathcal{X}} \left[\|x - f_\varphi(f_\theta(x))\|_2^2 \right] \longrightarrow \min_{\varphi, \theta}$$

На этом этапе логично задаться вопросом. Что будет в случае $d > D$?



Можно было подумать, что модель будет выучивать какие-то новые признаки в латентном представлении, которых не было в исходных объектах.

Но на самом деле единственная цель любой нейросети – минимизация лосса $\implies$ в таком сетепе модель скорее всего в каком-то виде скопирует x в латентное представление, а оставшаяся часть будет заполнена шумом, который модель будет полностью игнорировать.

Как все это применять и для чего? Например, автоэнкодеры можно использовать для визуализации многомерных данных: задав латентное представление двумерным или трехмерным, можно отрисовать объекты и глазками попытаться найти какие-то зависимости в данных.

Однако в общем, задача понижения размерности сама по себе является скорее вспомогательной, нежели самодостаточной. Маловероятна ситуация, при которой приходит заказчик и просит решить задачу понижения размерности.

1.4. Denoising Autoencoder

Более полезной постановкой, имеющей прямое практическое применение, будет задача расшумления данных. Например, на аудиозаписи голоса в какой-то момент появляется шум сверления у соседей, который хочется научиться убирать. Идея: будем подавать в энкодер зашумленные данные (аудиодорожка), а на выходе из декодера ожидать чистые данные (только голос).

Формально, пусть $\tilde{x}$ – зашумленный объект, x – целевой объект без шума, где

$$\tilde{x} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

$$\mathbb{E}_{x \sim \mathcal{X}} \left[\|x - f_{\varphi}(f_{\theta}(\tilde{x}))\|_2^2 \right] \longrightarrow \min_{\varphi, \theta}$$

Конкретно в этом примере шум генерируется нормальный, но все как обычно зависит от задачи. В данные, которые мы будем подавать энкодеру на обучении, нужно закладывать какой именно шум мы хотим восстанавливать.

Заметим, что для того, чтобы все это обучить, нам нужны чистые данные. Соответственно, придется один раз потратиться на сбор очень качественных примеров, на которые потом будет накладываться синтетический шум.

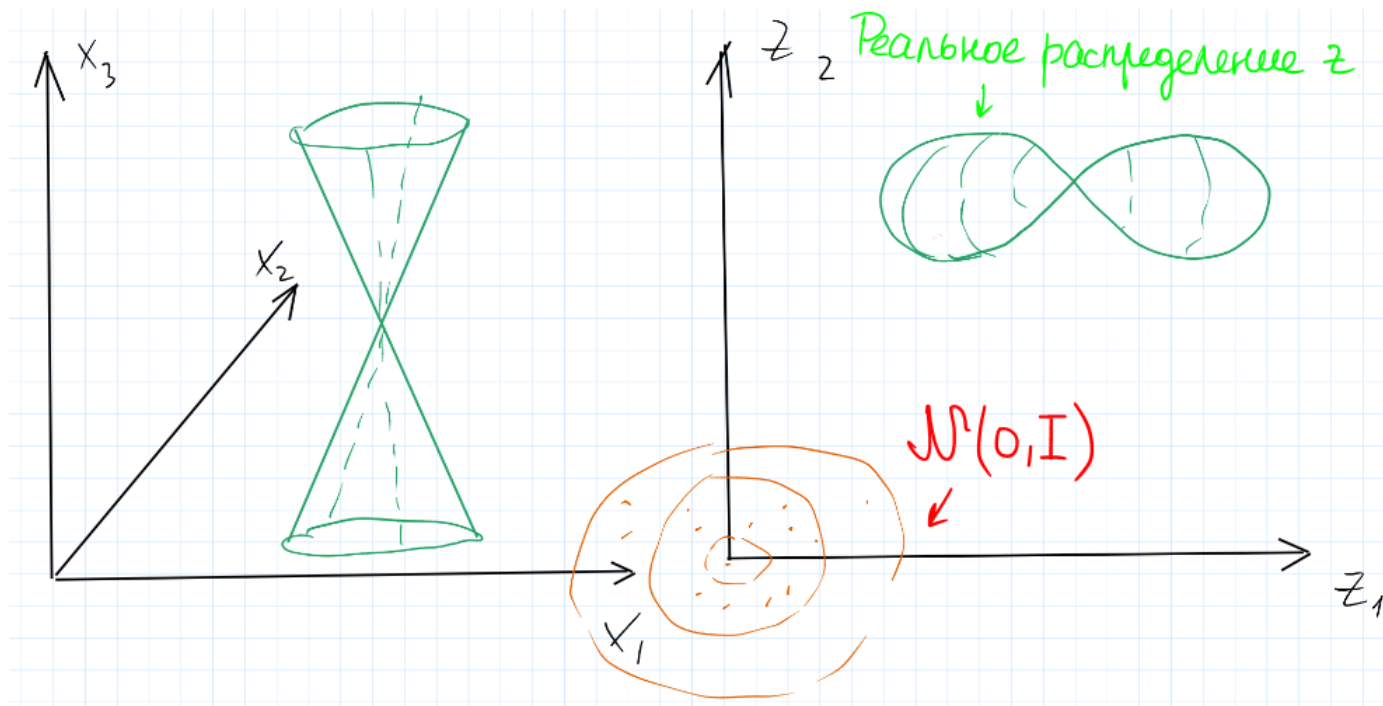
1.5. Семплирование

До сих пор обсуждались только supervised задачи, но изначально хотелось использовать автоэнкодер в качестве генеративной модели, которая умеет семплировать новые объекты.

Первая идея, которая приходит в голову: давайте оставим только декодер, а генерировать объекты будем следующим образом:

$$z \sim \mathcal{N}(0, I) \implies x_{gen} = f_{\varphi}(z)$$

К сожалению, такая вещь не заведется. Почему? Если мы возьмем z , который получился из энкодинга какого-то объекта, то декодер справится с некоторыми поправками восстановить исходный объект. Однако при обучении модели у нас нет никаких гарантий на вид распределения представлений, которые приходят декодеру.

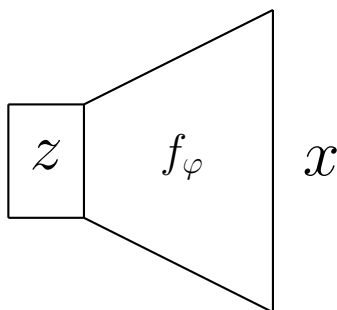


Кажется, что осталась всего одна проблема – нужно ввести ограничение на то, чтобы распределение на выходах энкодера (входе декодера) было нормальным. Тогда можно будет семплировать привычные для декодера z , на которые он учился, и получать новые примеры.

1.6. Variational Autoencoder (VAE)

Сейчас придется вспомнить модели с латентными переменными (привет ЕМ-алгоритму). Зададим модель на вероятностном языке, а затем поймем, как ее вообще обучать.

Для начала рассмотрим только декодер:



Запишем совместное распределение x и z при условии параметров φ как

$$p(x, z | \varphi) = p(x | z, \varphi) p(z | \varphi) := p(x | z, \varphi) p(z)$$

Первый **множитель** как раз отвечает за декодирование: действительно, выход декодера x зависит только от входа z и параметров нейросети декодера φ .

Второй **множитель** отвечает за распределение латентных переменных z , которые мы предполагаем не зависящими от параметров φ .

Эти распределения мы параметризуем следующим образом:

$$p(z) := \mathcal{N}(z | 0, I)$$

$$p(x | z, \varphi) := \mathcal{N}(x | \mu_\varphi(z), \Sigma_\varphi(z)), \quad \mu_\varphi(z) \in \mathbb{R}^D, \quad \Sigma_\varphi(z) \in \mathbb{R}^{D \times D}$$

Если говорить на языке ЕМ-алгоритма: x являются наблюдаемыми переменными, а z – латентными. Тогда нашей целью является максимизация следующего правдоподобия:

$$\log p(x | \varphi) = \log \int_{\mathbb{R}^d} p(x, z | \varphi) dz \longrightarrow \max_{\varphi}$$

Заметим, что под логарифмом, вообще говоря, интегрируется сложная нейросеть, то есть интеграл не берется. Честно максимизировать правдоподобие не получится, поэтому предлагается облегчить задачу путем вывода нижней вариационной границы (ELBO):

$$\begin{aligned} \log p(x | \varphi) &= \log p(x | \varphi) \cdot 1 = \left[\text{любое распределение } q(z) \right] = \log p(x | \varphi) \int_{\mathbb{R}^d} q(z) dz = \\ &= \int_{\mathbb{R}^d} \log p(x | \varphi) q(z) dz = \left[p(x | \varphi) = \frac{p(x, z | \varphi)}{p(z | x, \varphi)} \right] = \int_{\mathbb{R}^d} q(z) \log \frac{p(x, z | \varphi)}{p(z | x, \varphi)} dz = \\ &= \int_{\mathbb{R}^d} q(z) \log \frac{p(x, z | \varphi) q(z)}{p(z | x, \varphi) q(z)} dz = \int_{\mathbb{R}^d} q(z) \log \frac{p(x, z | \varphi)}{q(z)} + \int_{\mathbb{R}^d} q(z) \log \frac{q(z)}{p(z | x, \varphi)} = \\ &= \underbrace{\mathcal{L}(q, \varphi)}_{\text{ELBO}} + \text{KL} \left(q(z) \parallel p(z | x, \varphi) \right) \end{aligned}$$

Таким образом, правдоподобие представилось в виде суммы нижней вариационной оценки и KL-дивергенции между распределением q и апостериорным распределением $p(z | x, \varphi)$.

Варьируя распределение q , можно балансировать между двумя слагаемыми. Если положить распределение на z равным апостериорному при условии данных и параметров, получаем

$$q(z) := p(z | x, \varphi) \implies \text{KL} \left(q(z) \parallel p(z | x, \varphi) \right) = 0 \implies \mathcal{L}(q, \varphi) = \log p(x | \varphi)$$

Вспомним, что при выводе формулы выше распределение q может быть абсолютно произвольным. В частности, мы можем положить его равным

$$q(z) := q(z | x, \theta), \quad \text{где } \theta \text{ — параметры энкодера}$$

Это ключевое предположение, которое позволит сделать нашу модель именно автокодировщиком, а не абстрактной моделью с произвольными латентными переменными. То есть q как раз и будет кодировщиком, задающим распределение латентных переменных z при условии входного объекта x и параметров нейросети энкодера θ .

Таким образом, мы получили обе части автоэнкодера:

$$q(z) := q(z | x, \theta) \text{ — энкодер,} \quad p(x | z, \varphi) \text{ — декодер}$$

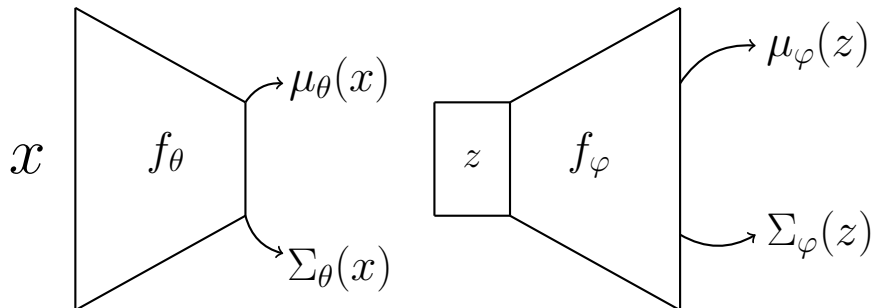
Перепишем ELBO для нового q , который и будет лоссом при обучении:

$$\mathcal{L}(q, \varphi) = \int_{\mathbb{R}^d} q(z | x, \theta) \log \frac{p(x, z | \varphi)}{q(z | x, \theta)} \longrightarrow \max_{\varphi, \theta}$$

Осталось определиться с видом распределения, которое задает q :

$$q(z | x, \theta) := \mathcal{N}(z | \mu_\theta(x), \Sigma_\theta(x))$$

В итоге вся архитектура выглядит следующим образом:



То есть энкодер предсказывает параметры распределения z , а декодер — распределения x .

Заметим, что матрицы $\Sigma_\theta(x)$, $\Sigma_\varphi(z)$ имеют размеры $d \times d$ и $D \times D$ соответственно, что очень много уже для каких-нибудь картинок адекватного качества. Поэтому будем предсказывать

$$\Sigma_\theta(x) = \text{diag}(\sigma_{\theta_1}^2(x), \dots, \sigma_{\theta_d}^2(x)), \quad \Sigma_\varphi(z) = \text{diag}(\sigma_{\varphi_1}^2(z), \dots, \sigma_{\varphi_D}^2(z))$$

Почему хватает диагональной матрицы? Казалось бы, в тех же картинках все соседние пиксели сильно коррелируют. На самом деле нелинейностей внутри энкодера и декодера уже достаточно для моделирования взаимодействий между компонентами, ведь конечное изображение получается из одного латентного вектора z , а значит в реальности пиксели будут скоррелированы. На практике можно даже положить $\Sigma_\varphi(z) := I$ и все равно будет работать.

Опять запишем полученный лосс:

$$\mathcal{L}(q, \varphi) := \mathcal{L}(\theta, \varphi) = \int_{\mathbb{R}^d} q(z | x, \theta) \log \frac{p(x, z | \varphi)}{q(z | x, \theta)} dz = (*)$$

Вспомним, что до этого мы задали совместное распределение как

$$p(x, z | \varphi) = p(x | z, \varphi) p(z)$$

Подставляя это выражение в лосс, получаем

$$\begin{aligned} (*) &= \int_{\mathbb{R}^d} q(z | x, \theta) \log \frac{p(x | z, \varphi) p(z)}{q(z | x, \theta)} dz = \\ &= \int_{\mathbb{R}^d} q(z | x, \theta) \log p(x | z, \varphi) dz + \int_{\mathbb{R}^d} q(z | x, \theta) \log \frac{p(z)}{q(z | x, \theta)} dz = \\ &= \underbrace{\mathbb{E}_{z \sim q(z | x, \theta)} \left[\log p(x | z, \varphi) \right]}_{\text{reconstruction loss (MSE)}} - \underbrace{\text{KL} \left(q(z | x, \theta) \parallel p(z) \right)}_{\text{regularization}} \end{aligned}$$

Заметим, что нормальные распределения были заданы не просто так.

- Во-первых, первая компонента лосса в терминах машинного обучения означает, что мы будем просто семплировать батч переменных z и усреднять по нему же. Заметим, что под матожиданием стоит логарифм плотности нормального распределения, а значит это будет тот самый MSE с некоторыми поправками.
- Во-вторых, распределения в KL-дивергенции в регуляризаторном слагаемом являются нормальными, а значит его можно посчитать аналитически и использовать при обучении.

По сути, reconstruction loss отвечает за качество восстановленных автоэнкодером объектов, а вторая часть лосса – регуляризатор, который пытается приблизить латентное распределение $q(z | x, \theta)$ к априорному распределению $p(z)$, которое мы задали как стандартное нормальное.

В сущности, мы решили ту самую проблему незнания латентного распределения на входе декодера, введя ограничение на его вид и таким образом заставив его быть стандартным нормальным. Теперь для генерации можно семплировать $z \sim \mathcal{N}(0, I)$ и прогонять через декодер.

Однако остается еще одна проблема: при обучении нам нужно считать градиент лосса по параметрам θ и φ . Со вторым проблем нет, так как φ есть только внутри матожидания и градиент спокойно прокидывается. Но как посчитать градиент reconstruction loss по θ , которая есть в параметрах распределения? По сути нужно посчитать градиент операции семплирования.

1.7. Reparameterization trick

Вспомним, следующий факт из теории вероятностей:

$$z \sim \mathcal{N}(z \mid \mu_\theta(x), \Sigma_\theta(x)), \varepsilon \sim \mathcal{N}(0, I) \implies z = \mu_\theta(x) + \Sigma_\theta(x)^{1/2} \varepsilon$$

А значит reconstruction loss можно переписать в виде

$$\mathbb{E}_{z \sim q(z \mid x, \theta)} \left[\log p(x \mid z, \varphi) \right] = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0, I)} \left[\log p(x \mid \mu_\theta(x) + \Sigma_\theta(x)^{1/2} \varepsilon, \varphi) \right]$$

Таким образом, мы вытащили параметры θ из распределения, по которому брались матожидания, и теперь градиенты можно спокойно считать как и для параметров φ .

Такая техника называется **reparameterization trick**. Заметим, что опять же все это удалось проверить благодаря хорошим свойствам нормального распределения.

1.8. Вывод формулы итогового лосса

Посчитаем reconstruction loss для одного z :

$$\begin{aligned} \log p(x \mid z, \varphi) &= \log \left(\frac{1}{(2\pi)^{D/2} |\Sigma|^{1/2}} \exp \left\{ -\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right\} \right) = \\ &= -\frac{D}{2} \log 2\pi - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) = \text{const}(\mu, \Sigma) - \frac{1}{2} \sum_{i=1}^D \sigma_i^2 - \frac{1}{2} \sum_{i=1}^D \frac{(x_i - \mu_i)^2}{\sigma_i^2} \end{aligned}$$

Если для декодера взять $\Sigma_\varphi(z) = I$, получим

$$\log p(x \mid z, \varphi) \propto -\frac{1}{2} \sum_{i=1}^D (x_i - \mu_i)^2 \implies -\frac{1}{2} \text{MSE},$$

где x_i – исходный объект, а μ_i – выход декодера

Теперь распишем слагаемое, отвечающее за регуляризацию. Для этого вспомним, что

$$\begin{aligned} \xi_1 &\sim \mathcal{N}(\xi_1 \mid \mu_1, \Sigma_1), \quad \xi_2 \sim \mathcal{N}(\xi_2 \mid \mu_2, \Sigma_2) \\ \implies \text{KL}(\xi_1 \parallel \xi_2) &= \frac{1}{2} \left[\text{tr}(\Sigma_2^{-1} \Sigma_1) - d + (\mu_2 - \mu_1)^T \Sigma_2^{-1} (\mu_2 - \mu_1) + \ln \frac{|\Sigma_2|}{|\Sigma_1|} \right] \end{aligned}$$

Таким образом, полагая $\Sigma_\theta = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$:

$$\text{KL} \left(q(z \mid x, \theta) \parallel p(z) \right) = \frac{1}{2} \left[\text{tr}(\Sigma_\theta) - d + \|\mu_\theta\|_2^2 - \log \Sigma_\theta \right] = -\frac{d}{2} - \frac{1}{2} \sum_{i=1}^d (\sigma_i^2 + \mu_i^2 - \log \sigma_i^2)$$

Для того, чтобы всегда получать неотрицательные значения σ_i^2 , будем предсказывать $\log \sigma_i^2$, а затем экспоненцировать это значение для получения дисперсии.

1.9. Заключение

Сравнивая с GANами, вариационный автоэнкодер решает проблему mode collapse, от которой сильно страдают первые. Сэмплы получаются более разнообразные, покрывается все распределение данных.

С другой стороны, картинки в GANах получаются очень хорошего качества, максимально реалистичные. Вариационный автоэнкодер в свою очередь генерирует более размытые изображения, которые легко отличить от реальных. Это происходит из-за приближения пикселей нормальным распределением, что не соответствует действительности.

Вывод: для генерации картинок, где важно качество, и mode collapse не так критичен, стоит использовать GANы. Если же для задачи очень важно покрывать все разнообразие распределения, как в биоинформатике или физике, разумнее будет выбрать VAE.